

the microcontroller's attention, the traditional approach of polled-loop driver fails to recognise the user inputs. This requirement causes a serious problem when the user is attempting to configure the data and traffic is flooding over networking (in this case serial) interface.

(b) If the MCU is busy in performing the sensor data processing, either using the off-chip ADC interface option or on-chip ADC register polling, and the user is triggering an input using the input device. In this case also, the user attention will be unanswered as the MCU is already busy in sensory task.

Polled mode 4x3 embedded C driver and its validation for C51 MCU

Let us consider the 4x3 matrix keypad with four rows and three columns being connected to Port 1 of AT89S52 MCU, as shown in Fig. 1. This is basically the connections in Proteus simulation software. The four LEDs are used only for testing and are connected to P2.0 through P2.3. A typical keypad layout diagram is shown in Fig. 2. The polled-loop key-scanning algorithm is very simple and straightforward, as

shown in steps 1 through 7 given below:

Step-1: Configure the columns as inputs (logic-1) and rows as outputs (logic-0)
 Step-2: Start polling the KeyScan by looking for any change in column (1 to 0)
 Step-3: Is there any change in columns (i.e., C1=0 or C2=0 or C3=0)

If yes,
 then, go to Step - 4.

else
 Go to Step-2.

Step-4: Make R1=0 and keep R2=R3=R4=1 and,

If C1=0 then,
 pressed key is - '1' and return.
 else if C2=0 then,
 pressed key is - '2' and return.
 else if C3=0 then,
 pressed key is - '3' and return.
 else
 go to step - 5:

Step-5: Make R2=0 and keep R1=R3=R4=1 and,

If C1=0 then,
 pressed key is - '4' and return.
 else if C2=0 then,
 pressed key is - '5' and return.
 else if C3=0 then,
 pressed key is - '6' and return.
 else
 go to step - 6:

Step-6: Make R3=0 and keep R1=R2=R4=1 and,

If C1=0 then,
 pressed key is - '7' and return.
 else if C2=0 then,
 pressed key is - '8' and return.
 else if C3=0 then,
 pressed key is - '9' and return.
 else
 go to step - 7

Step-7: Make R4=0 and keep R1=R2=R3=1 and,

If C1=0 then,
 pressed key is - '*' and return.
 else if C2=0 then,
 pressed key is - '0' and return.
 else if C3=0 then,
 pressed key is - '#' and return.
 else
 go to step - 2

```
1. unsigned char KeyScan(void)
2. {
3.     R1=R2=R3=R4=0;
4.     C1=C2=C3=1;
5.     while (1)
6.     {
7.         MSDelay(20); //This delay is used to avoid key debouncing.
8.         if((C1==0||C2==0||C3==0)) //Is there any key is pressed?
9.         {
10.            R1=0; R2=R3=R4=1;
11.            if(C1==0) return '1'; //ASCII('1')=0x31 in Hex.
12.            if(C2==0) return '2';
13.            if(C3==0) return '3';
14.
15.            R2=0; R1=R3=R4=1;
16.            if(C1==0) return '4';
17.            if(C2==0) return '5';
18.            if(C3==0) return '6';
19.
20.            R3=0; R1=R2=R4=1;
21.            if(C1==0) return '7';
22.            if(C2==0) return '8';
23.            if(C3==0) return '9';
24.
25.            R4=0; R1=R2=R3=1;
26.            if(C1==0) return '*';
27.            if(C2==0) return '0';
28.            if(C3==0) return '#';
29.        }
30.    }
31. }
```

Fig. 3: Source listing for polled-loop key scanning implemented in embedded-C

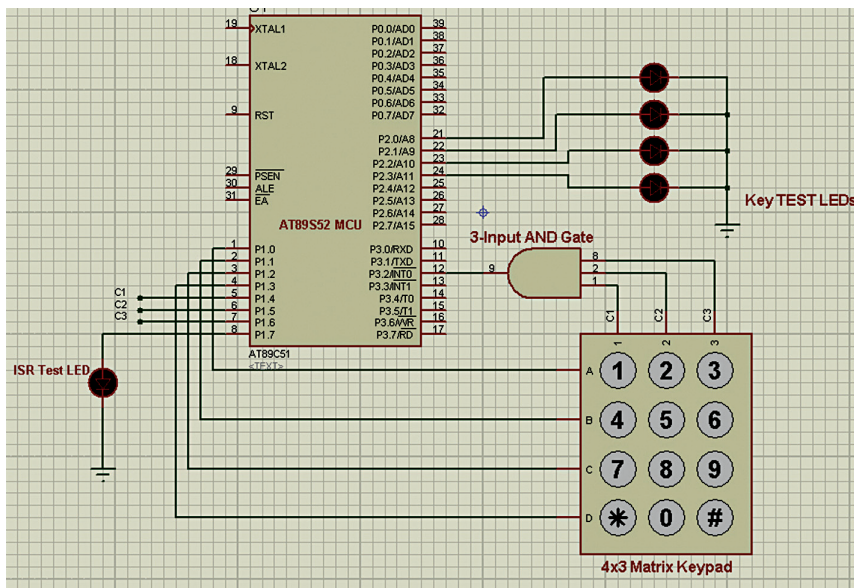


Fig. 4: Circuit diagram for interrupt driven keypad